

```

import numpy as np
# Sequence: x1, x2, x3
X = [1.0, 2.0, 3.0]
# Weight matrices (scalar case for simplicity)
Wx = 0.5 # input weight
Wh = 0.8 # hidden weight
b = 0.0 # bias

# Initial hidden state
h_prev = 0.0
def tanh(x):
    return np.tanh(x)
hidden_states = []

print("Step-by-step RNN computation:\n")

for t, x_t in enumerate(X):
    h_t = tanh(Wx * x_t + Wh * h_prev + b)

    print(f"Time step {t+1}:")
    print(f"Input x_{t+1} = {x_t}")
    print(f"h_{t+1} = tanh({Wx}*{x_t} + {Wh}*{h_prev}) = {h_t:.4f}\n")

    hidden_states.append(h_t)
    h_prev = h_t
    print("Final Hidden States:", hidden_states)

import matplotlib.pyplot as plt

plt.plot(hidden_states, marker='o')
plt.title("Hidden State Evolution Over Time")
plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.show()

```

Step-by-step RNN computation:

Time step 1:

Input $x_1 = 1.0$

$h_1 = \tanh(0.5 \cdot 1.0 + 0.8 \cdot 0.0) = 0.4621$

Final Hidden States: [np.float64(0.46211715726000974)]

Time step 2:

Input $x_2 = 2.0$

$h_2 = \tanh(0.5 \cdot 2.0 + 0.8 \cdot 0.46211715726000974) = 0.8786$

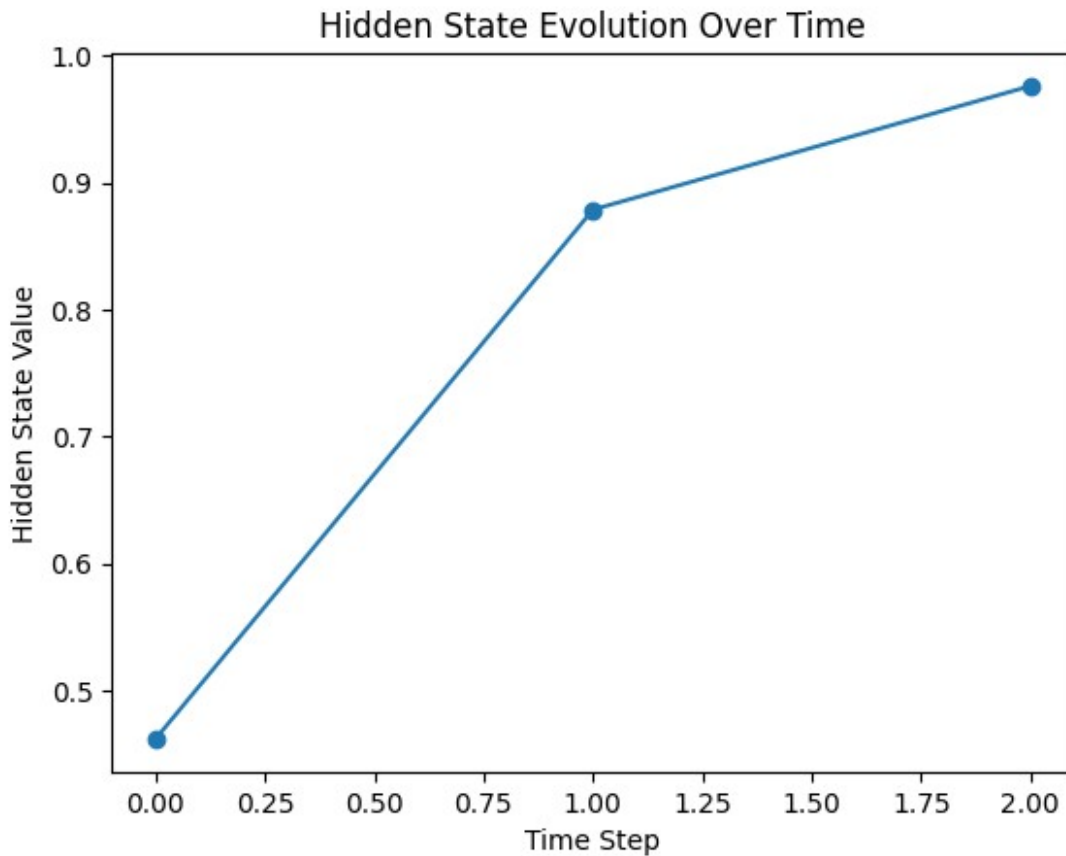
Final Hidden States: [np.float64(0.46211715726000974),
np.float64(0.8786223746838898)]

Time step 3:

Input $x_3 = 3.0$

```
h_3 = tanh(0.5*3.0 + 0.8*0.8786223746838898) = 0.9759
```

```
Final Hidden States: [np.float64(0.46211715726000974),  
np.float64(0.8786223746838898), np.float64(0.9758816208890569)]
```



```
import numpy as np
import matplotlib.pyplot as plt

# Modified sequence (task 1: changed number/value of inputs)
X = [3.0, 4.0, 5.0, 7.0]

# Modified parameters (task 2)
Wx = 0.6 # input weight (changed)
Wh = 0.8 # hidden weight
b = 0.2 # bias (changed)

# Initial hidden state
h_prev = 0.0

def tanh(x):
    return np.tanh(x)

hidden_states = []
```

```

print("Step-by-step RNN computation with modified inputs/parameters:\n")

for t, x_t in enumerate(X):
    h_t = tanh(Wx * x_t + Wh * h_prev + b)

    print(f"Time step {t+1}:")
    print(f"Input x_{t+1} = {x_t}")
    print(f"h_{t+1} = tanh({Wx}*{x_t} + {Wh}*{h_prev} + {b}) = {h_t:.4f}\n")

    hidden_states.append(h_t)
    h_prev = h_t

print("Final Hidden States:", hidden_states)

plt.plot(hidden_states, marker='o')
plt.title("Hidden State Evolution (Modified Inputs & Parameters)")
plt.xlabel("Time Step")
plt.ylabel("Hidden State Value")
plt.grid(True)
plt.show()

```

Step-by-step RNN computation with modified inputs/parameters:

Time step 1:

Input $x_1 = 3.0$

$h_1 = \tanh(0.6*3.0 + 0.8*0.0 + 0.2) = 0.9640$

Time step 2:

Input $x_2 = 4.0$

$h_2 = \tanh(0.6*4.0 + 0.8*0.9640275800758169 + 0.2) = 0.9976$

Time step 3:

Input $x_3 = 5.0$

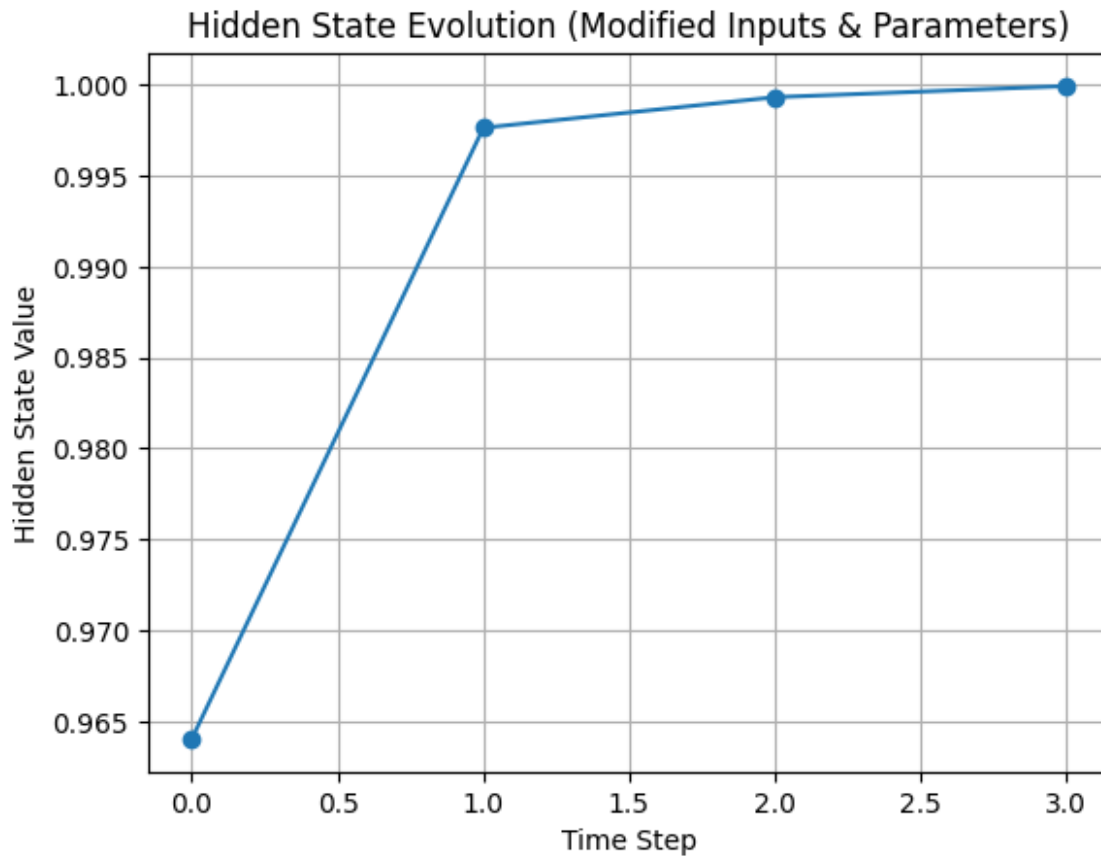
$h_3 = \tanh(0.6*5.0 + 0.8*0.997643260097823 + 0.2) = 0.9993$

Time step 4:

Input $x_4 = 7.0$

$h_4 = \tanh(0.6*7.0 + 0.8*0.9993267667520128 + 0.2) = 0.9999$

Final Hidden States: [np.float64(0.9640275800758169),
np.float64(0.997643260097823), np.float64(0.9993267667520128),
np.float64(0.9999390712927598)]



```
import numpy as np

# -----
# 1. Input sequence
# -----
X = [1.0, 2.0, 3.0]
target = 0.5 # desired output

# -----
# 2. Initialize parameters
# -----
Wx = 0.5
Wh = 0.8
Wy = 1.0
b = 0.0

# Learning rate
lr = 0.01

# -----
# 3. Activation functions
# -----
def tanh(x):
```

```

        return np.tanh(x)

def dtanh(x):
    return 1 - np.tanh(x)**2

# -----
# 4. Forward Pass
# -----
h = [0.0]    # h0 = 0

for t in range(len(X)):
    h_t = tanh(Wx * X[t] + Wh * h[t] + b)
    h.append(h_t)

# Output (many-to-one)
y_pred = Wy * h[-1]

# Loss (Mean Squared Error)
loss = 0.5 * (y_pred - target)**2

print("Forward Pass:")
print("Hidden states:", h[1:])
print("Prediction:", y_pred)
print("Loss:", loss)

# -----
# 5. Backward Pass (BPTT)
# -----

# Gradients initialization
dWx = 0
dWh = 0
dWy = 0
db = 0

# Output gradient
dy = (y_pred - target)

# Gradient for Wy
dWy = dy * h[-1]

# Backprop through time
dh_next = dy * Wy    # gradient flowing into last hidden state

for t in reversed(range(len(X))):
    # derivative of tanh
    dt = dtanh(Wx * X[t] + Wh * h[t] + b)

    # current gradient
    dh = dh_next * dt

```

```

print(f"\nTime step {t}")
print("dh:", dh)
# accumulate gradients
dWx += dh * X[t]
dWh += dh * h[t]
db += dh

# pass gradient backward in time
dh_next = dh * Wh
# PRINT AFTER UPDATING dh_next
print("dh_next:", dh_next)
print("\nGradients:")
print("dWx:", dWx)
print("dWh:", dWh)
print("dWy:", dWy)
print("db:", db)

# -----
# 6. Update weights
# -----
Wx -= lr * dWx
Wh -= lr * dWh
Wy -= lr * dWy
b -= lr * db

print("\nUpdated Parameters:")
print("Wx:", Wx)
print("Wh:", Wh)
print("Wy:", Wy)
print("b:", b)

```

Forward Pass:

Hidden states: [np.float64(0.46211715726000974),
np.float64(0.8786223746838898), np.float64(0.9758816208890569)]
Prediction: 0.9758816208890569
Loss: 0.11323165854999806

Time step 2

dh: 0.02267816815333795
dh_next: 0.01814253452267036

Time step 1

dh: 0.0041369101186258935
dh_next: 0.003309528094900715

Time step 0

dh: 0.0026027708674217116
dh_next: 0.0020822166939373694

Gradients:

dWx: 0.07891109556468734

dWh: 0.02183728310022592

dWy: 0.4644041275445246

db: 0.029417849139385555

Updated Parameters:

Wx: 0.4992108890443531

Wh: 0.7997816271689978

Wy: 0.9953559587245547

b: -0.00029417849139385554

```
import numpy as np
```

```
# -----
```

```
# 1. Input sequence
```

```
# -----
```

```
X = [3.0, 4.0, 5.0, 7.0]
```

```
target = 0.5 # desired output
```

```
# -----
```

```
# 2. Initialize parameters
```

```
# -----
```

```
Wx = 0.5
```

```
Wh = 0.8
```

```
Wy = 1.0
```

```
b = 0.0
```

```
# Learning rate
```

```
lr = 0.01
```

```
# -----
```

```
# 3. Activation functions
```

```
# -----
```

```
def tanh(x):
```

```
    return np.tanh(x)
```

```
def dtanh(x):
```

```
    return 1 - np.tanh(x)**2
```

```
# -----
```

```
# 4. Forward Pass
```

```
# -----
```

```
h = [0.0] # h0 = 0
```

```
for t in range(len(X)):
```

```
    h_t = tanh(Wx * X[t] + Wh * h[t] + b)
```

```
    h.append(h_t)
```

```
# Output (many-to-one)
```

```

y_pred = Wy * h[-1]

# Loss (Mean Squared Error)
loss = 0.5 * (y_pred - target)**2

print("Forward Pass:")
print("Hidden states:", h[1:])
print("Prediction:", y_pred)
print("Loss:", loss)

# -----
# 5. Backward Pass (BPTT)
# -----

# Gradients initialization
dWx = 0
dWh = 0
dWy = 0
db = 0

# Output gradient
dy = (y_pred - target)

# Gradient for Wy
dWy = dy * h[-1]

# Backprop through time
dh_next = dy * Wy # gradient flowing into last hidden state

for t in reversed(range(len(X))):

    # derivative of tanh
    dt = dtanh(Wx * X[t] + Wh * h[t] + b)

    # current gradient
    dh = dh_next * dt
    print(f"\nTime step {t}")
    print("dh:", dh)
    # accumulate gradients
    dWx += dh * X[t]
    dWh += dh * h[t]
    db += dh

    # pass gradient backward in time
    dh_next = dh * Wh
    # PRINT AFTER UPDATING dh_next
    print("dh_next:", dh_next)
print("\nGradients:")
print("dWx:", dWx)
print("dWh:", dWh)

```



```

print("dWy:", dWy)
print("db:", db)

# -----
# 6. Update weights
# -----
Wx -= lr * dWx
Wh -= lr * dWh
Wy -= lr * dWy
b  -= lr * db

print("\nUpdated Parameters:")
print("Wx:", Wx)
print("Wh:", Wh)
print("Wy:", Wy)
print("b:", b)

```

Forward Pass:

```

Hidden states: [np.float64(0.9051482536448665),
np.float64(0.9914291183462953), np.float64(0.9972454952330853),
np.float64(0.9996302304239262)]
Prediction: 0.9996302304239262
Loss: 0.12481518357673278

```

Time step 3

```

dh: 0.0003694278027837851
dh_next: 0.00029554224222702813

```

Time step 2

```

dh: 1.6259026634547525e-06
dh_next: 1.3007221307638021e-06

```

Time step 1

```

dh: 2.220111983050913e-08
dh_next: 1.7760895864407306e-08

```

Time step 0

```

dh: 3.2095117959299705e-09
dh_next: 2.5676094367439764e-09

```

Gradients:

```

dWx: 0.0025942225658184795
dWh: 0.0003700422746889757
dWy: 0.4994454823654287
db: 0.0003710791160788663

```

Updated Parameters:

```

Wx: 0.49997405777434184
Wh: 0.7999962995772532

```

Wy: 0.9950055451763458
b: -3.7107911607886633e-06

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
```

```
vocab_size = 5000
max_len = 200
embed_size=32
hidden_size=32
```

```
(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=vocab_size)
```

```
X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)
```

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embed_size,
input_length=max_len),
    SimpleRNN(hidden_size),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

```
model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=2,
    verbose=1
)
```

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy*100:.2f}%")
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>

17464789/17464789 ————— 0s 0us/step
Epoch 1/20

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
embedding.py:100: UserWarning: Argument `input\_length` is deprecated.
Just remove it.

warnings.warn(

12500/12500 ————— 391s 31ms/step - accuracy: 0.6930 -
loss: 0.5720

Epoch 2/20

12500/12500 ————— 454s 32ms/step - accuracy: 0.7150 -
loss: 0.5524

Epoch 3/20

12500/12500 ————— 394s 32ms/step - accuracy: 0.7654 -
loss: 0.4890

Epoch 4/20

12500/12500 ————— 385s 31ms/step - accuracy: 0.7975 -
loss: 0.4414

Epoch 5/20

12500/12500 ————— 388s 31ms/step - accuracy: 0.7832 -
loss: 0.4634

Epoch 6/20

12500/12500 ————— 388s 31ms/step - accuracy: 0.8024 -
loss: 0.4304

Epoch 7/20

12500/12500 ————— 389s 31ms/step - accuracy: 0.8286 -
loss: 0.3909

Epoch 8/20

12500/12500 ————— 449s 32ms/step - accuracy: 0.8326 -
loss: 0.3832

Epoch 9/20

3008/12500 ————— 5:03 32ms/step - accuracy: 0.8442 -
loss: 0.3650